Programozási tételek I.
Alapok

A programozási tételek alatt a gyakran használt algoritmusokat értjük. Ezek a következők:

- Összegzés
- Megszámlálás
- Keresés
- Kiválogatás
- Maximum
- Minimum
- Rendezések

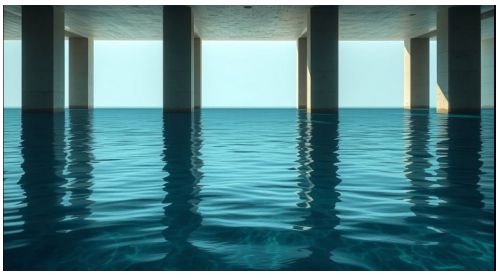
Az első szekcióban az alapokra szorítkozom. A további részekben részletesebben kitérek a nagyobb fejezetekre, mint a keresések és a rendezések. Az első és legfontosabb kérdés: *Miért kell ezeket tudni, ha a legtöbb programozási nyelvben vannak beépített függvények, amik elvégzik helyettünk ezeket?* Mert egy algoritmus struktúráját ezeken keresztül látjuk meg. Az elemi műveletek megismerésének segítségével felismerhetjük, hogy egy esetleges implementációban egyszerűsíteni tudunk egy beépített függvény segítségével. Ha nem ismerjük az algoritmusokat, de használni tudjuk a beépített függvényeket, az előbb utóbb a kódminőségünk rovására fog menni, mert vagy nem látjuk meg a kódrészletben az algoritmust, vagy egyszerűen felesleges kódokat írunk.
Ezen algoritmusok közelebb visznek minket az „elemi művelet” fogalmának megértéséhez is.

Miért nem egyértelmű a hétköznapokban egy maximum keresés algoritmusának az implementációja? Az evolúció kegyes volt hozzánk abban a tekintetben, hogy a kifinomult receptoraink a másodpercek törtrésze alatt feldolgozza a bejövő adatokat, és felismerjük mondjuk a maximumot. De vajon mindig olyan jól működik ez a rendszer?

Példa: Ha látunk 3-4 különböző méretű narancsot, akkor jó eséllyel 1-2 másodpercen belül meglátjuk a legnagyobbat. Természetesen ez az eljárás azonos 30-40 narancs esetében is, csak a műveletek száma akkor már több, így kicsit lelassulunk.

Példa: Mivel ismerjük a számok szimbólumait és a helyiértékes számábrázolás csodáját, ezért vélhetően a következő számsorból sem esik nehezünkre a minimum kiválasztása: 4,5,97,3,83,1000

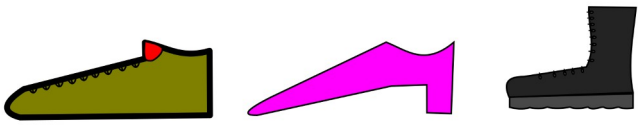
Példa: Az űrtartalomra már nehezebben tudunk saccolni. Lássuk a következő 3 képet:



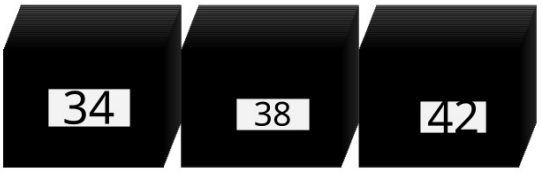
1. kép: AI képek a víztárolókról

Ahhoz, hogy meg tudjuk állapítani melyik víztartályban van a legtöbb víz, szükségünk van a térfogatra, valamint az oszlopok térfogatára. Ezek még nem alapadatok, hiszen a térfogat számításához is szükségünk van adatokra, mint oszlopok száma, alapjaiknak a területe, magassága, víztároló szélessége, hosszúsága, magassága, valamint víznek a szintje. Azokat nevezzük alapadatoknak, amelyeket már tovább nem tudunk bontani, az alapműveleteket ezen végezve jutunk a további adatokhoz. Amikor algoritmust tervezünk, akkor mindig ezekkel az alapadatokkal kezdjük a munkát.

Talán nehéznek tűnhet elsőre az algoritmusok megértése, ezért fontos megértenünk „mit látunk mi” és „mit lát a gép”. Tegyünk magunk elé 3 különböző méretű cipőt. Vélhetően a másodperc tört része alatt megállapítjuk, hogy melyik a legnagyobb. Ezt látjuk mi az agyunkkal. Tegyük bele ezeket a cipőket dobozokba, és írjuk rá a dobozokra, hogy melyik dobozban hányas cipőt helyeztük el. A legnagyobbat már nem a méret alapján találjuk meg, hanem a dobozra írt számok alapján. Itt a már említett számok szimbólumának ismerete segíti a dolgunkat. Hiszen a cipőt már nem látjuk, csak azokat a jeleket, amik a számunkra fontos információt tartalmazzák: a cipőméreteket. Ezt is az agyunkkal látjuk, felhasználva a korábbi ismereteinket (felismerjük a számokat). Mit lát a gép?
Most tegyük át a cipőket olyan dobozokba amiken semmilyen szám nem szerepel, és ugyanúgy néznek ki. Az ideális esetben „ennyit lát a gép”. És ebből kell építkeznünk amikor algoritmusokat tervezünk. Most eljutottunk oda, hogy a rendelkezésünkre álló információkat lecsökkentettük arra a szintre, ahol már az evolúciós előnyünket nem tudjuk felhasználni. A kifinomult receptorainknak gátat szab, hogy 150 és 156 doboz esetében már „ránézésre” a számukat sem tudjuk megmondani.
Nézzük ezt képekkel:



Kép 2: Ránézésre megállapítható a méretbeli különbség.



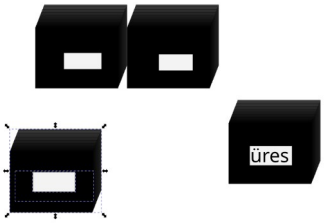
Kép 3: Mivel ismerjük a számokat, szintén ránézésre megállapítható a különbség

Kép 4: Amikor szeretnénk egy listát kezelni valahogyan, a számítógép nagyjából „ennyit tud”

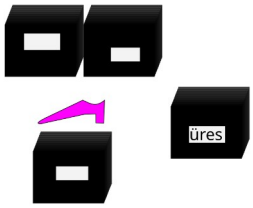
Minimum

Most hogy tudjuk, mit lát a gép, elkezdhetünk filozofálni az algoritmusokon. A cipős példa adja nekünk minimum maximum keresés algoritmusát. Nézzük először képeken, hogy mit kéne

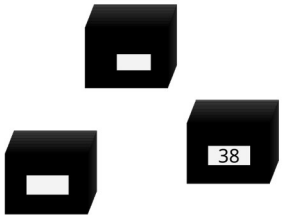
csinálnunk.
A célunk, hogy az üres dobozban a legkisebb cipő legyen. Ez lesz az algoritmusunk eredménye.



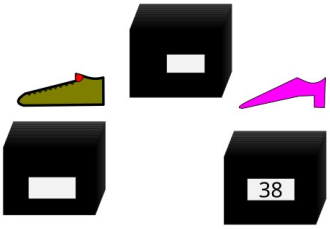
Fogom az első dobozt, nem tudok erről semmit, alávetem egy vizsgálatnak. Tehát kinyitom a dobozt.



Ebben egy 38-as cipő van, a dobozkám üres, szóval nincs szükség vizsgálatra. Belerakom a dobozba.

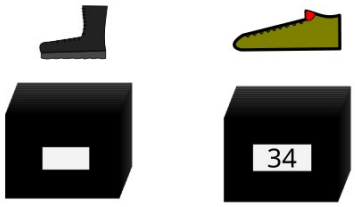


Fogom a következő dobozt, és kiveszem gyorsan, mivel a dobozban már van valami ezért vizsgálatnak vetem alá a kettő doboz tartalmát.



Összevetem a két cipő méretét (34, 38), és a kisebbet beleteszem a dobozba. Esetünkben történik egy csere.

Az utolsó dobozt is kiveszem vizsgálatra. Ismét úgy kell eljárnom, hogy összevetem a két cipő méretét.

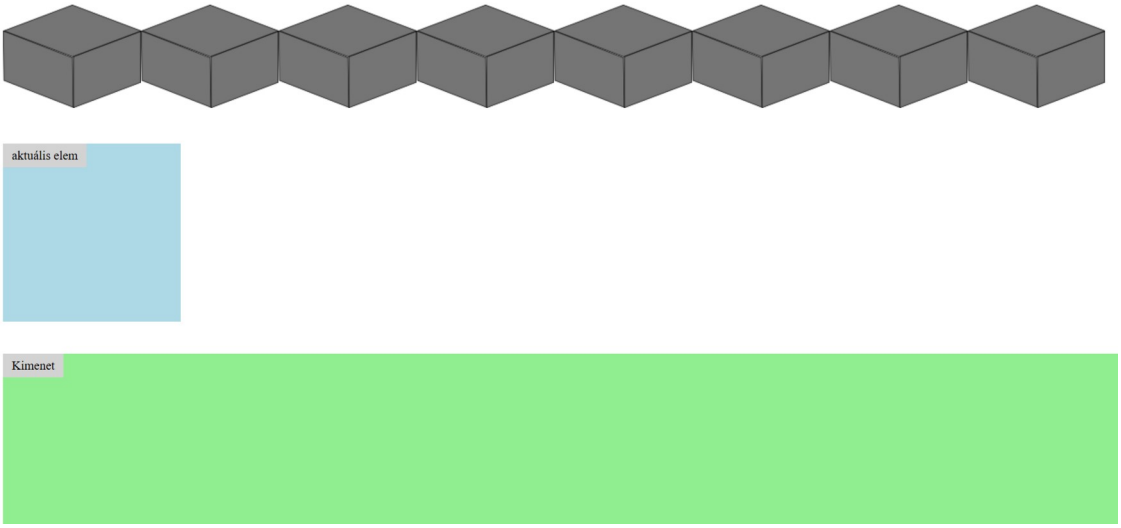


Összehasonlításunk alkalmával ezúttal nem kell kicserélnünk a cipőt a dobozunkban, mert egy nagyobb méretű cipőt találtunk a dobozban (42). Tehát a minimumot megtaláltuk, hiszen időközben végigértünk a teljes halmazon, amelynek az elemeit hasonlítgattuk.

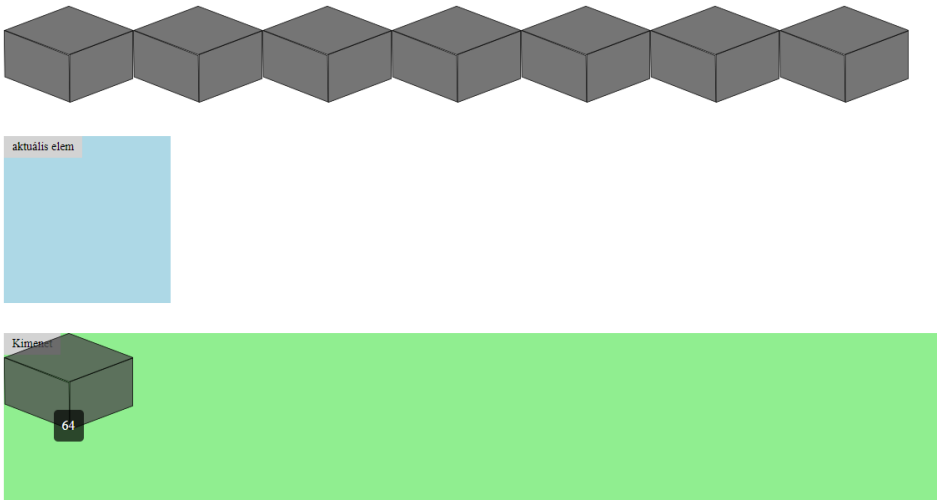
Nézzük pszeudokóddal az algoritmust:
min=t[0] // ez az amikor még üres a dobozunk és vizsgálat nélkül beletettük az elemünket.
ciklus i = 1 .. n-1 // a többi elemnél már ahogyan láttuk is, ugyanazt fogjuk csinálni
  ha t[i] < min // cipőket hasonlítgatunk, mindig pontosan 2 darabot vetünk össze, ha kisebb az új elem a dobozunk tartalmánál, akkor
    min = t[i] // beledobjuk a cipőt a dobozunkba
ciklus vége
ki min // megmutatjuk mi van a dobozunkban

A további algoritmusokat most már egyszerűsíteni tudjuk ábrázolásban, hiszen tudjuk, hogy egy szám egy tárgyat jelöl.

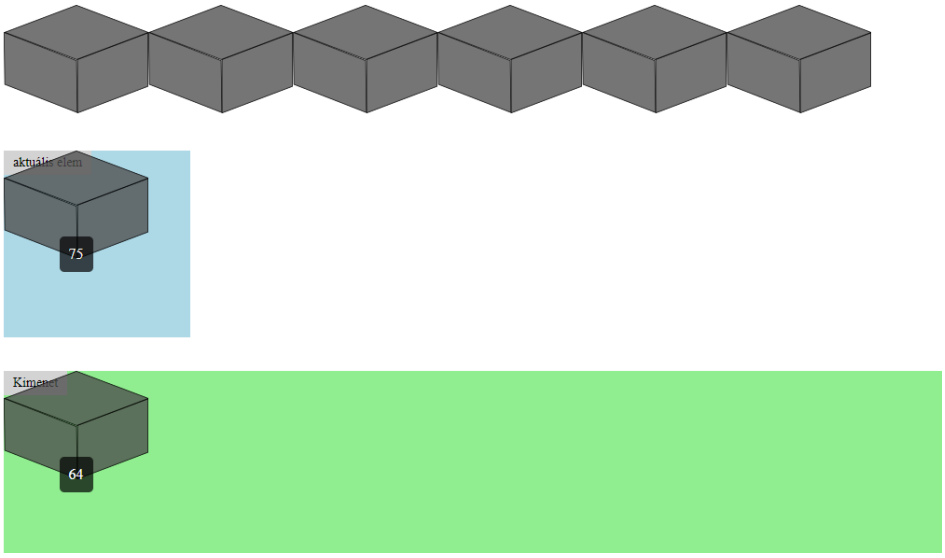
Maximum
A kiindulás most is hasonló, nézzük meg több elemmel, ezúttal kizárólag csak számok vannak a dobozainkban:



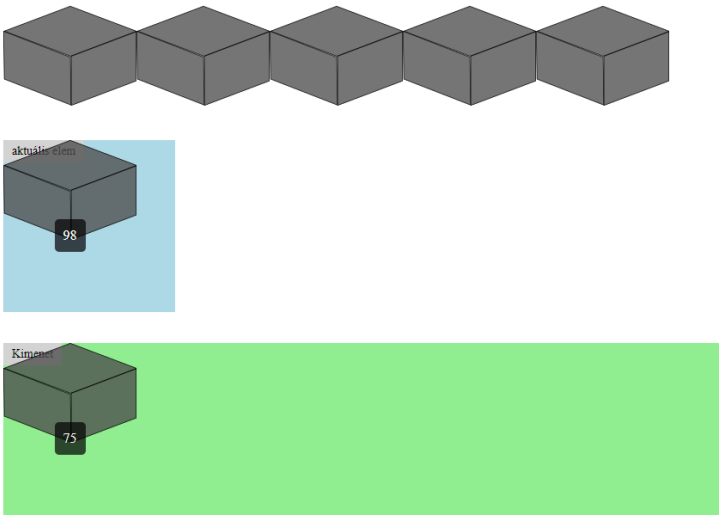
1. Felsorakozott a 8 doboz. A lehetséges kimenetnek célszerű itt is az első elemet azonnal megadni, de használhatjuk a MIN_VALUE konstansot is, amit sok programozási nyelvben megtalálhatunk. (js-ben: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/MIN_VALUE c#-ban pl.: https://learn.microsoft.com/en-us/dotnet/api/system.int32.minvalue?view=net-9.0)



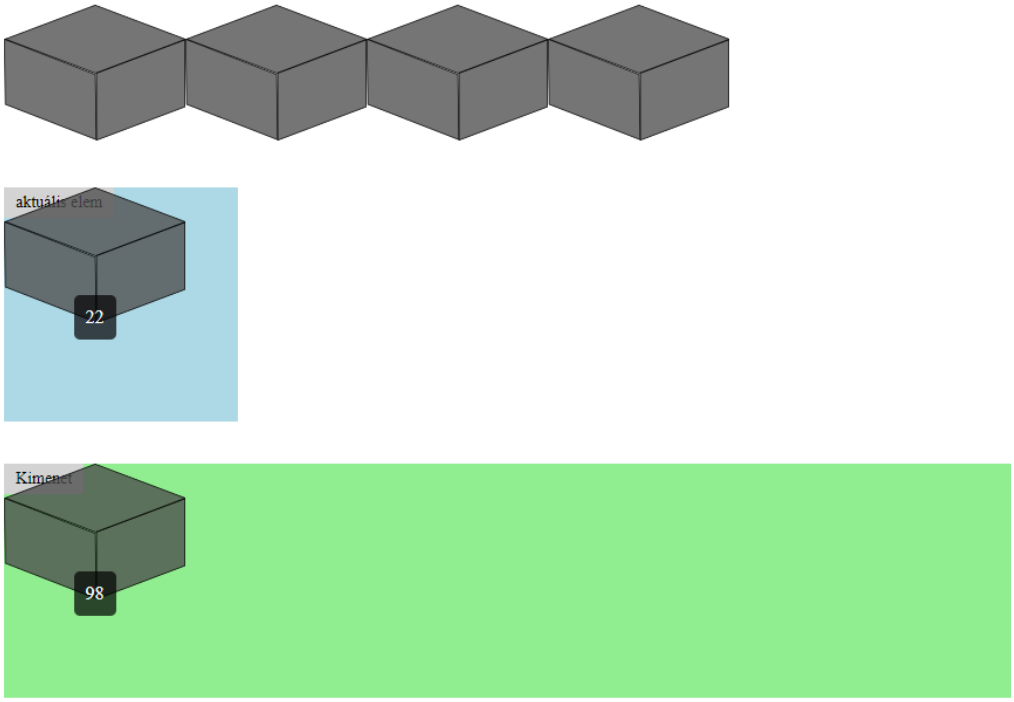
2. Az első elem a 64. Mivel még van pár elemünk, ezért itt már ciklust fogunk használni, hogy egyesével megvizsgáljuk az elemeket.



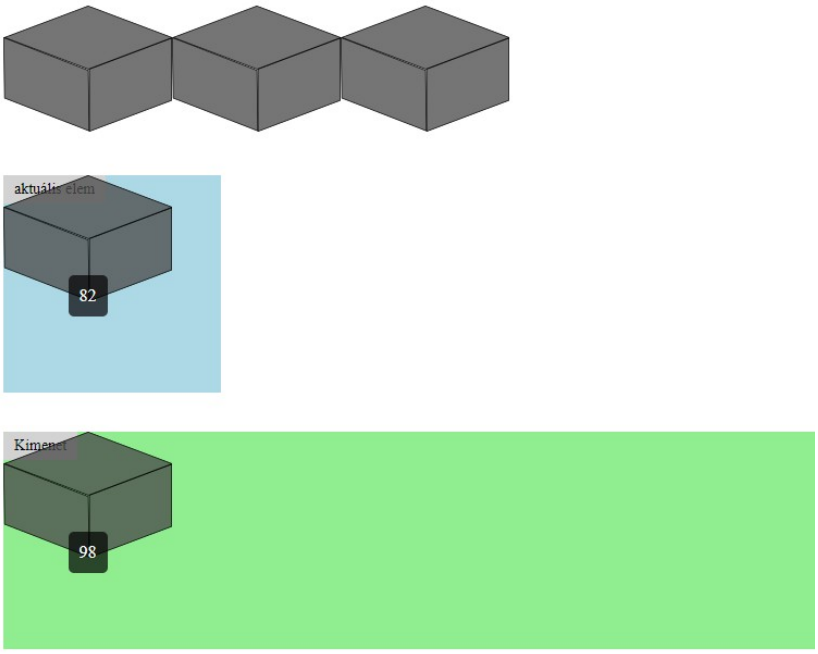
3. Belépünk a ciklusunkba, levesszük az aktuális (második) elemünket. Megnézzük nagyobb-e az épen aktuális kimenetnek gondoltnál. Mivel nagyobb, ezért a jelenlegi elemünket lecseréljük. A következő iterációban már az új elemünkhöz fogunk hasonlítani.



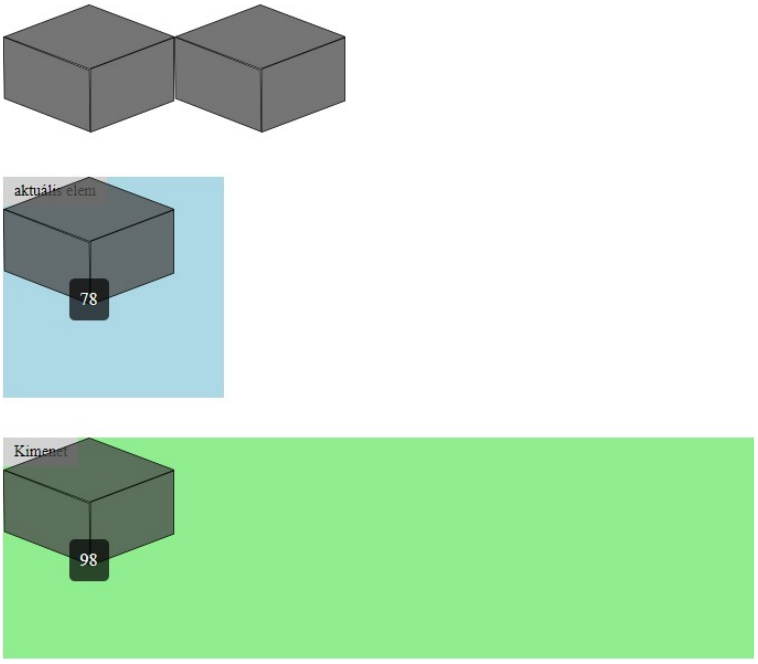
4. Levesszünk egy újabb elemet a tömbünkből. Ismét összehasonlítjuk, és mivel megint nagyobb elemmel találkoztunk, ezért hasonlóan járunk el, mint ahogy az előző lépésnél tettük. Jöhet is a következő iteráció.



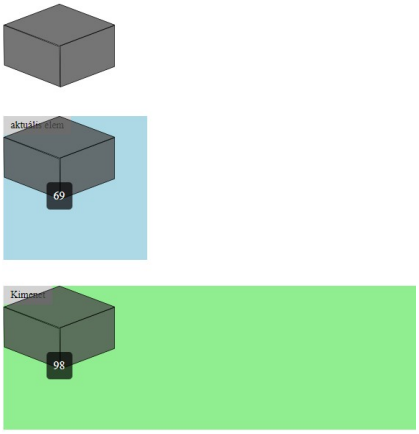
5. Leveszünk egy újabb elemet, és mivel ez kisebb, mint amit korábban már kimenetnek feltételeztünk, ezért nem hajtunk végre cserét.



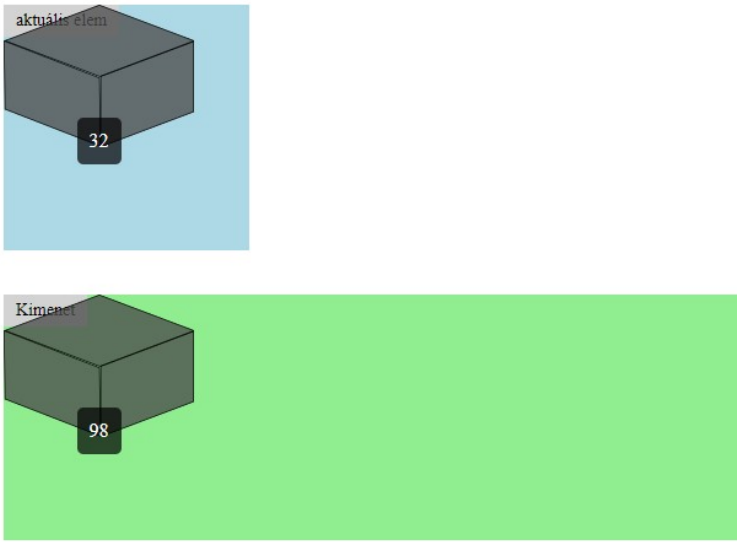
6. Leveszünk egy újabb elemet, összehasonlítjuk az elemeinket, és mivel kisebb, ezért nem hajtunk végre cserét.



7. Leveszünk ismét egy elemet, összehasonlítjuk megint a kimenetnek feltételezett elemmel, és mivel ismét kisebb elemünk van, nem hajtunk végre cserét.



8. Ismét egy újabb elemet vetünk alá a vizsgálatnak, majd mivel megint nem nagyobb az elem, ezért nem hajtunk végre cserét.



9. Természetesen az utolsó elemmel is elvégezzük az eddigi vizsgálatunkat, ismételten nem hajtunk végre cserét.

Ugyan a harmadik elem után már nem hajtottunk végre cserét, mivel nem tudtuk mi van a többi dobozban, ezért végig kellett mennünk a tömbön. Miután végigértünk a tömbön, a kimenettel visszatér az algoritmusunk.

pszeudokód:
max=t[0] // ez az amikor még üres a dobozunk és vizsgálat nélkül beletettük az elemünket.
ciklus i = 1 .. n-1 // a többi elemnél már ahogyan láttuk is, ugyanazt fogjuk csinálni
    ha t[i] > max // számokat hasonlítgatunk, mindig pontosan 2 darabot vetünk össze, ha nagyobb az új elem a dobozunk tartalmánál, akkor
        max = t[i] // lecseréljük a kimenet dobozának az értékét
ciklus vége
ki max // megmutatjuk mi van a dobozunkban

js kód:

```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
let maxValue = Number.MIN_VALUE;
for(const number of numbers){
    if(maxValue < number){
        maxValue = number;
    }
}

console.log(`Maxvalue: ${maxValue}`)
```

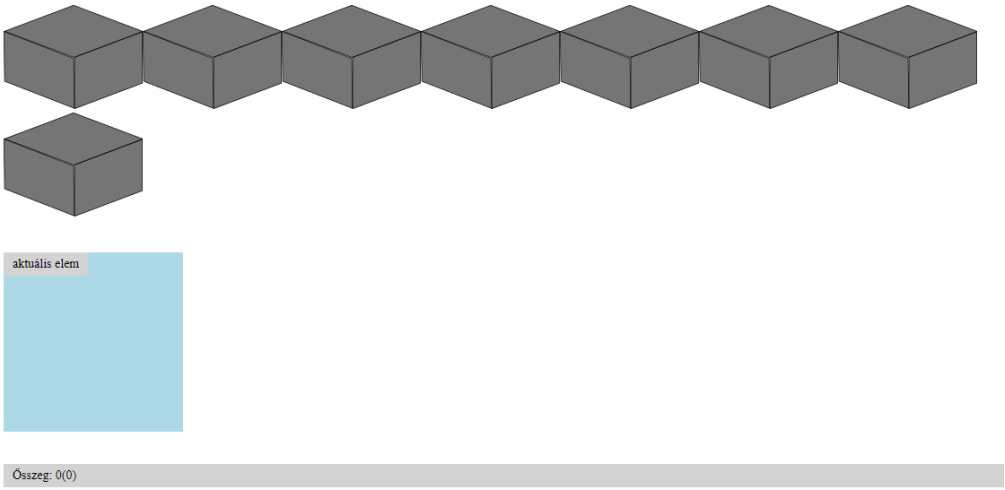
```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
let maxValue = numbers[0];
for(let i = 1;  i< numbers.length; i++){
    if(maxValue < numbers[i]){
        maxValue = numbers[i];
    }
}

console.log(`Maxvalue: ${maxValue}`)
```

```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
let maxValue = Number.MIN_VALUE;
for(let i = 0;  i< numbers.length; i++){
    if(maxValue < numbers[i]){
        maxValue = numbers[i];
    }
}

console.log(`Maxvalue: ${maxValue}`)
```

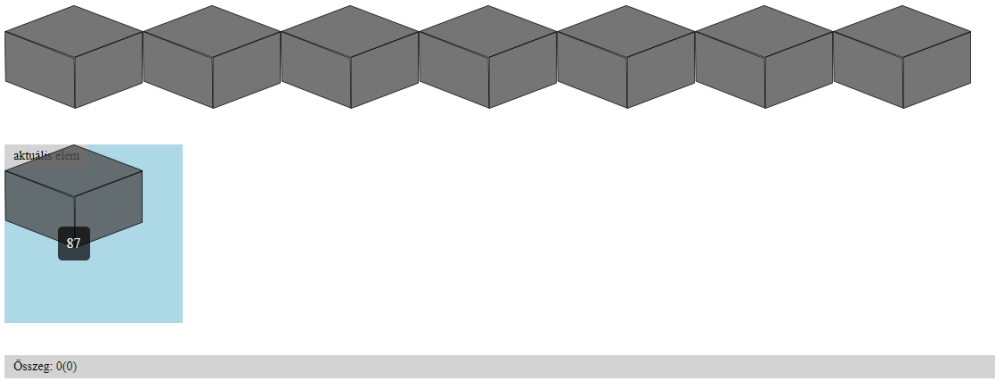
```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
let maxValue = Number.MIN_VALUE;
for(const index in numbers){
    if(maxValue < numbers[index]){
        maxValue = numbers[index];
    }
}

console.log(`Maxvalue: ${maxValue}`)
```

Összegzés

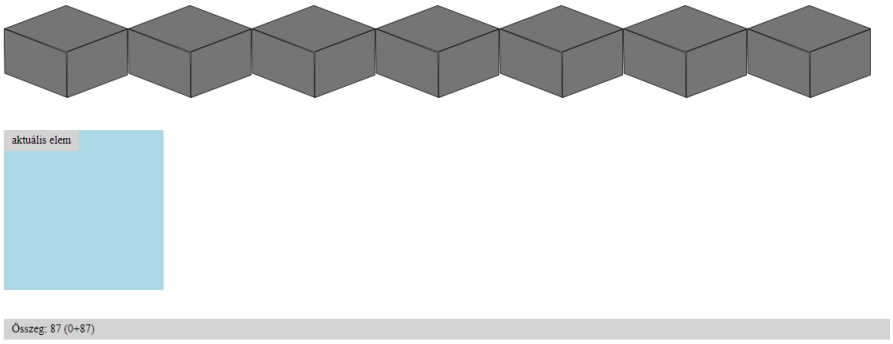
A megszámlálás esetén is hasonlóan járunk el, mint a korábbiaknál, viszont itt nem elemeket, hanem, értéket tárolunk el. Az algoritmus működése annyiban különbözik a korábbiaktól, hogy az alap algoritmusnál nem kötjük feltételhez, hogy milyen elemeknek az értékét összegezzük. Természetesen tetszés szerint bővíthető bármilyen feltételekkel.
Nézzük az algoritmust, először képekben.



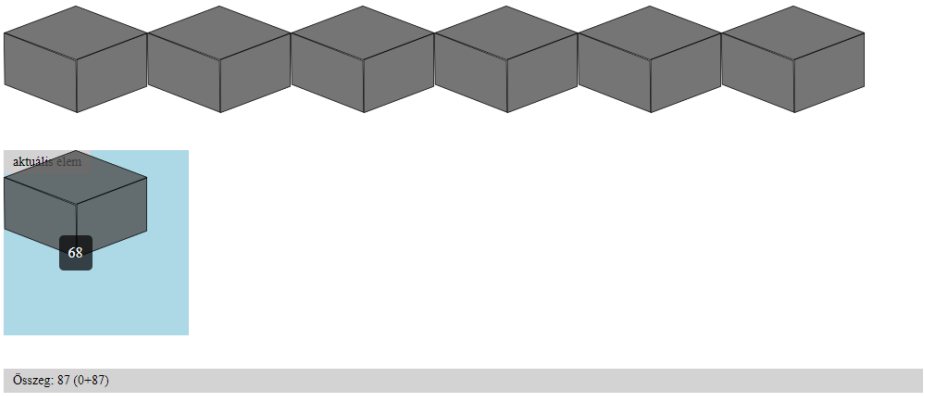
1. Ismét felsorakozott a 8 dobozkánk. Természetesen ismételten nem tudjuk mik vannak bennük. Mivel tudjuk, hogy összegzünk, és a dobozokat egyesével fogjuk megnézni, ezért létrehozunk egy összeg változót, aminek kezdetben nem meglepő módon 0 értéke van.



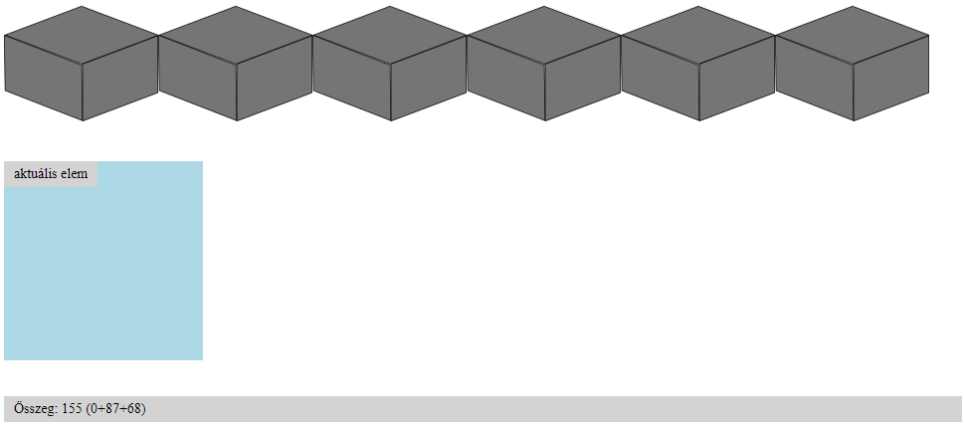
2. Leveszünk egy dobozt, megnézzük mi van benne.



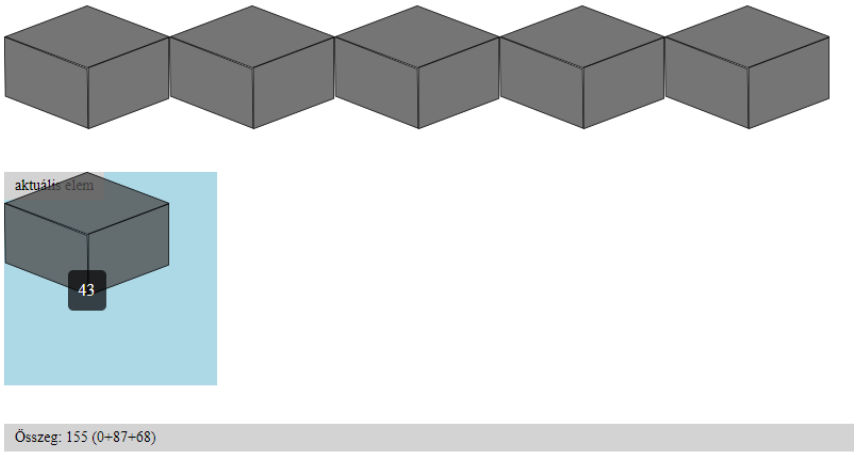
3. Szépen hozzáadjuk az összeghez, majd várjuk a következő dobozt.



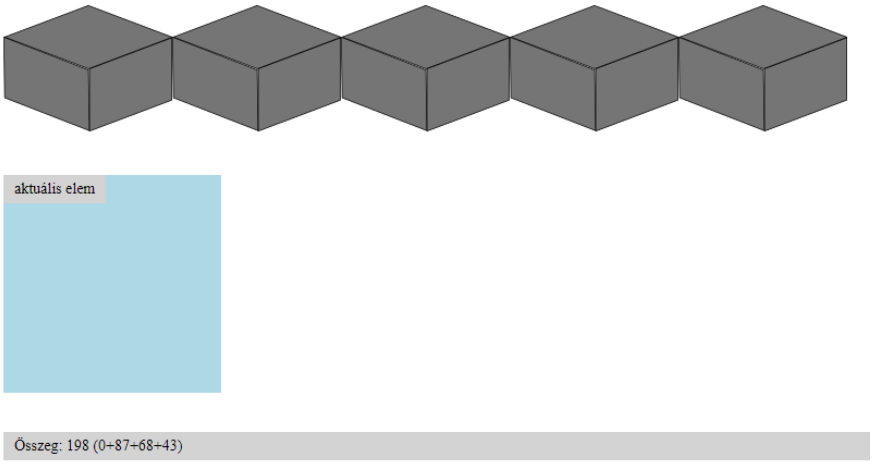
4. A következő doboz a 68.



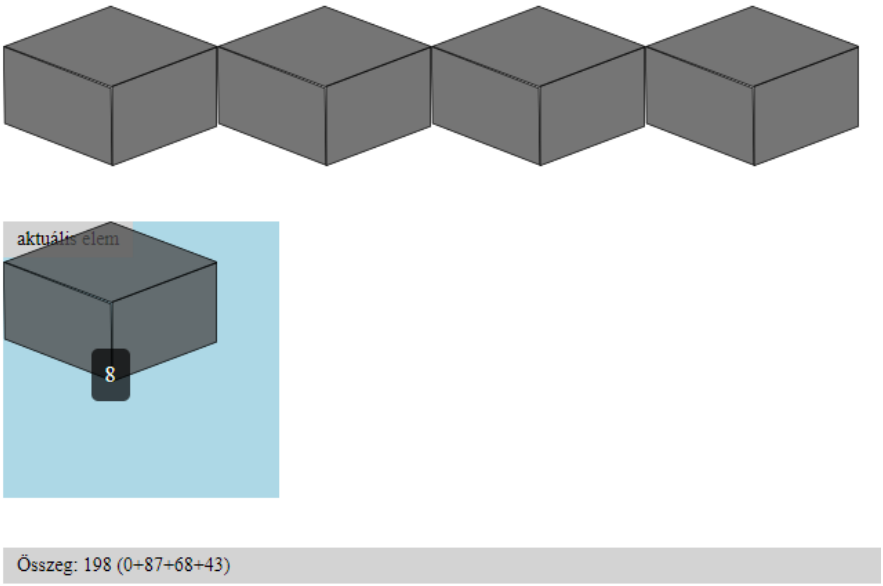
5. Hozzáadjuk az összeghez újra, várjuk a következő dobozt.



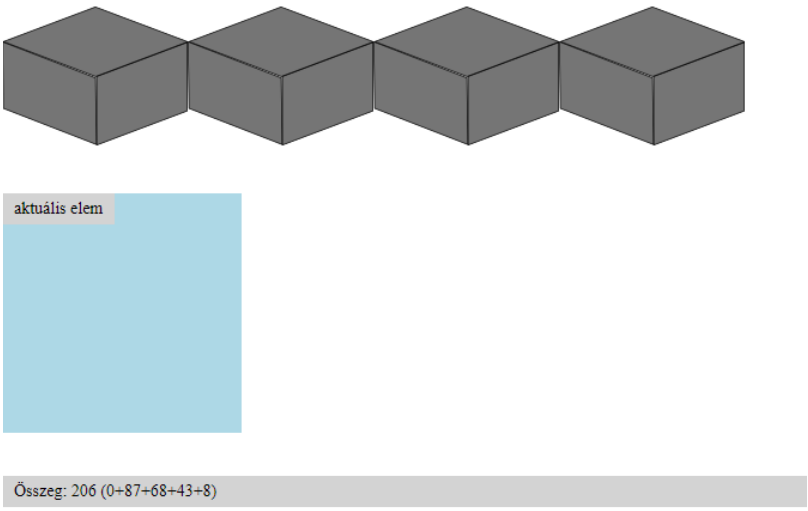
6. A következő dobozt is levesszük.



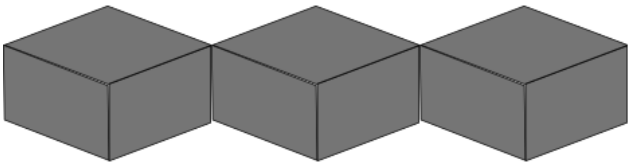
7. Hozzáadjuk ezt is.

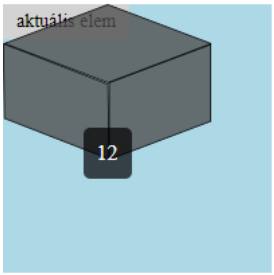


8. Leveszünk egy dobozt.



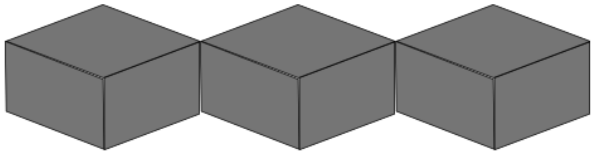
9. Hozzáadjuk





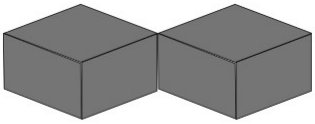
Összeg: 206 (0+87+68+43+8)

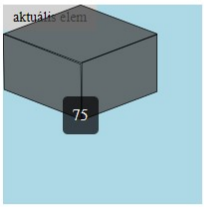
10. Leveszünk egy dobozt.



Összeg: 218 (0+87+68+43+8+12)

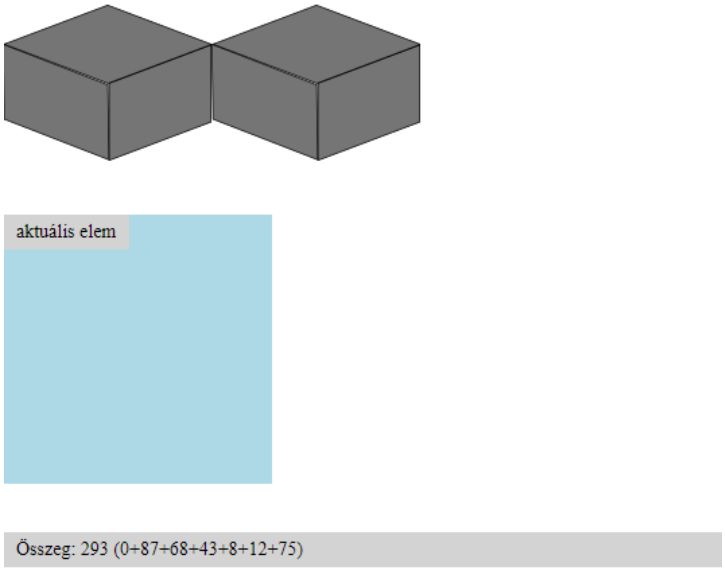
11. Hozzáadjuk



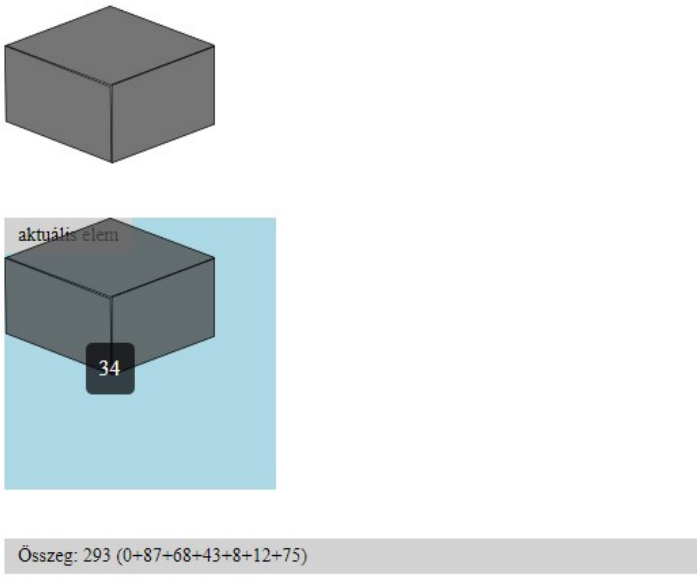


Összeg: 218 (0+87+68+43+8+12)

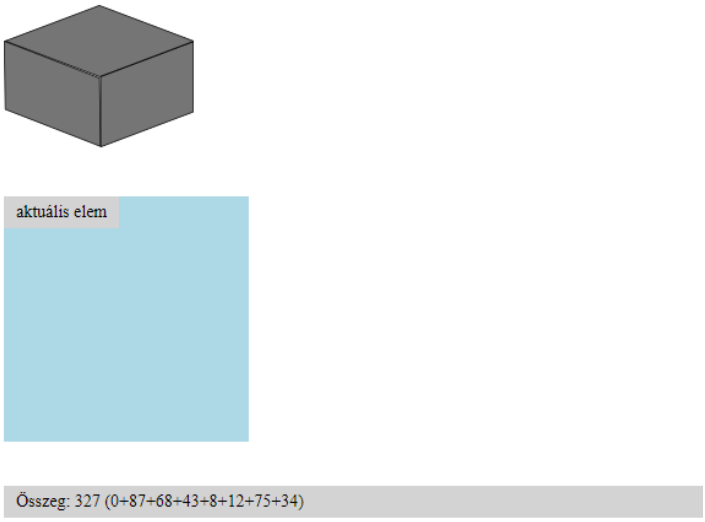
12. Leveszünk egy dobozt.



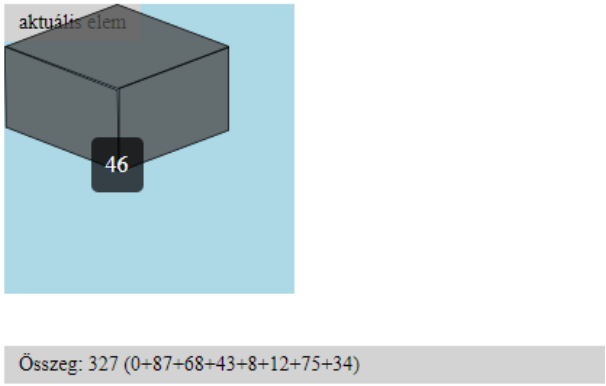
13. Hozzáadjuk



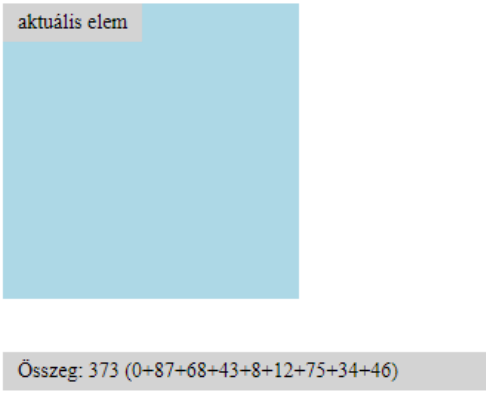
14. Leveszünk egy dobozt.



15. Hozzáadjuk



16. Levesszük az utolsó dobozt.



17. Végül ezt is szépen hozzáadjuk. Mivel nincs több elemünk, ezért be is fejeződik az algoritmusunk és kiadja nekünk az összeget. A 17 lépésben láthattuk, hogy melyek az ismételt tevékenységek, és az algoritmizálás ereje itt már nagyon szépen előjön, ha vetünk egy pillantást rá.

pszeudokód:
osszeg = 0 // inicializáljuk az összeget
ciklus i = 0 .. n -1 // ahogyan azt láthattuk, a tömb minden egyes eleménél ugyanazt fogjuk tenni
  osszeg = osszeg + t[i] // a tömb aktuális elemének az értékét hozzáadjuk az összeghez
ciklus vége
ki osszeg // visszatérünk az összeggel

js kód:

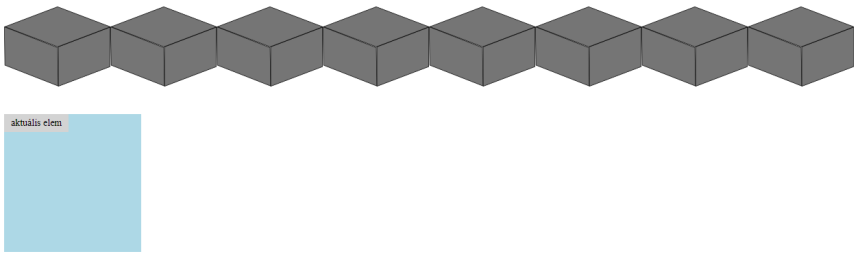
```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
let osszeg = 0;
for(const index in numbers){
    osszeg += numbers[index];
}

console.log(`Összeg: ${osszeg}`)
```

```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
let osszeg = 0;
for(let i = 0; i< numbers.length; i++){
    osszeg += numbers[i];
}

console.log(`Összeg: ${osszeg}`)
```

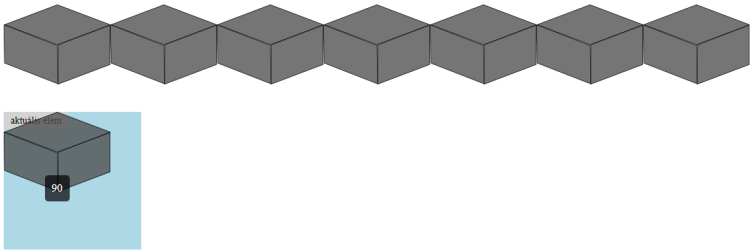
```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
let osszeg = 0;
for(const number of numbers){
    osszeg += number;
}

console.log(`Összeg: ${osszeg}`)
```

Gyakorlás
Feladat1: Számoljuk ki egy szám tömb elemeinek az átlagát!
Feladat2: Írjuk ki egy személyeket tartalmazó tömb (név, életkor) legfiatalabb és legidősebb tagját. Számoljuk ki az átlagéletkort!

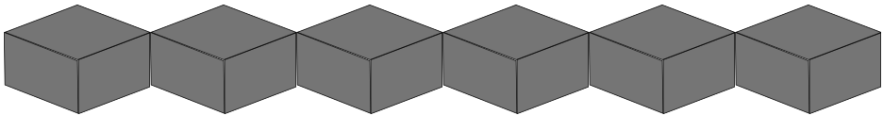
Keresés
A keresésnél megkülönböztetünk rendezett és rendezetlen adathalmazt. Sokszor a rendezetlen adathalmazt átalakítjuk más struktúrákra, hogy a keresés egyszerűbb legyen. A mesterséges intelligencia fejlődése az ilyen adatok struktúrába rendezésének köszönhető (nyilván a tárolókapacitás is egy szűk keresztmetszet, de most szoftveres oldalról közelítünk). Első körben nézzünk meg egy rendezetlen adathalmazban történő keresést. Szekvenciális (vagy másnéven lineáris) keresést fogunk alkalmazni erre.
A feladatunk ezúttal az, hogy megkeressük a dobozaink között azt, amelyben a 18-as szám van. Nézzük meg képekkel először:

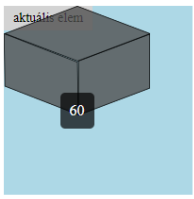


1. Ismételten a 8 dobozzal kezdünk, és egyesével kell megvizsgálnunk az elemeket.

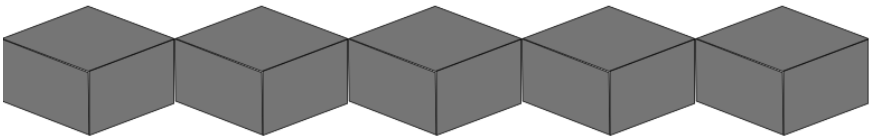


2. Megnézzük az első dobozt, láthatóan nem 18 az értéke. Így megyünk is tovább a második dobozzal.

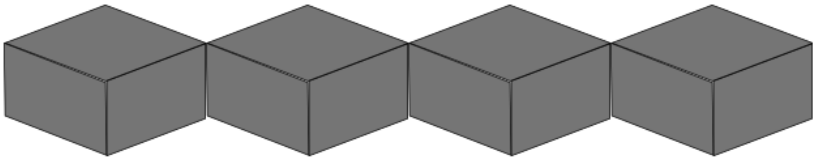


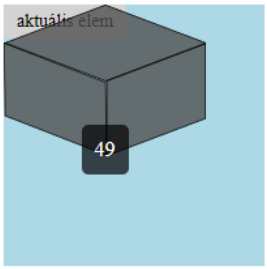


3. A második doboz szintén nem 18, megyünk tovább.

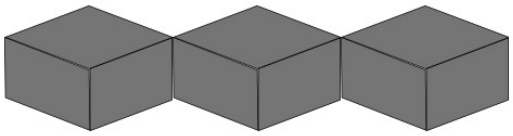


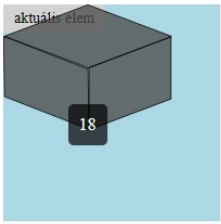
4. A harmadik dobozunk a 67-es számot tartalmazza, ami nem 18 még mindig. Megyünk tovább.





5. A negyedik dobozunk sem nyert.





6. Az ötödik dobozunk végre a 18-as. Így az algoritmusunk a végére ért. Azt is megtudtuk, hogy az 5. doboz tartalmazza a számot. Amikor tömbökben keresünk, akkor gyakran objektumok között végezzük a keresést, és egy tulajdonság alapján akarjuk megtudni, hogy a talált elem egy másik tulajdonságának mi az értéke. Például: Keressük meg egy személyeket tartalmazó tömbben az első olyan személyt, aki 2014-ben született, és írjuk ki a nevét. Természetesen az algoritmusunkat ilyen esetekben is maradéktalanul tudjuk használni.

pszeudokód:
ker = <tetszőleges érték> // az az érték amit keresünk
i = 0 // az aktuális elemszámunk.
ciklus amíg i<n és t[i]<>ker // addig megyünk, amíg meg nem találjuk a számot, vagy végig nem érünk a tömbön. Az utolsó elemünknél amikor még n-nél kisebb indexet nézünk, növeljük az indexet, így n-el egyenlő értékű lesz i. Mivel az i<n feltétel nem teljesül, kilépünk a ciklusból.
  i = i + 1 // i-t növeljük
ciklus vége // ciklus vége

Ha i<n akkor // ha i értéke n, akkor végigértünk a tömbünkön, és nem találtuk meg az elemet. Ha az utolsó elemet kerestük volna, akkor i értéke n-1, ezért ez a feltétel.
  ki "Van ilyen" // algoritmusunk kimenetele
  ki: "Indexe: ", i // Mivel minden algoritmusnak pontosan 1 kimenetele van, ezért általában ha nem találunk elemet, akkor -1-el térünk vissza egyébként meg az indexel. A pszeudokód szabadsága megengedi nekünk hogy két értékkel lépjünk ki igaz esetén.
különben
  ki: "A keresett érték nem található" //itt szoktunk -1-et adni eredményül
ha vége
Egy valós programban a kimenetet vizsgálatnak vetjük alá, és az alapján dolgozzuk fel kimenetet.

jskód:

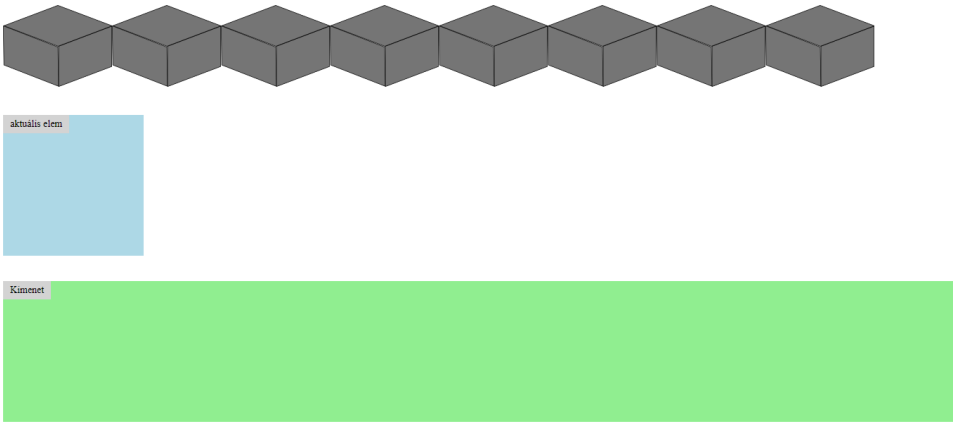
```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
const targetNumber = 7;
let i = 0;
while(i<numbers.length && numbers[i] != targetNumber){
    i++
}

if(i<numbers.length){
    console.log(`A keresett szám indexe: ${i}`)
}else{
    console.log(`Nincs ilyen szám a tömbben`)
}
```

```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
const targetNumber = 7;
let targetIndex = -1;
for(let i = 0; i<numbers.length; i++){
    if(numbers[i] == targetNumber){
        targetIndex =i;
        break;
    }

}


if(-1 < targetIndex){
    console.log(`A keresett szám indexe: ${targetIndex}`)
}else{
    console.log(`Nincs ilyen szám a tömbben`)
}
```

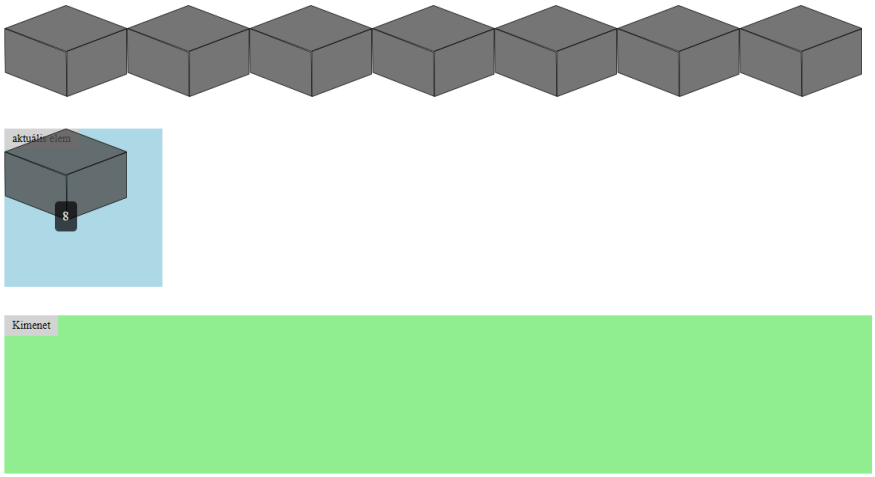
```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
const targetNumber = 7;
let targetIndex = -1;
for(const i in numbers){
    if(numbers[i] == targetNumber){
        targetIndex =i;
        break;
    }

}


if(-1 < targetIndex){
    console.log(`A keresett szám indexe: ${targetIndex}`)
}else{
    console.log(`Nincs ilyen szám a tömbben`)
}
```

Megszámlálás

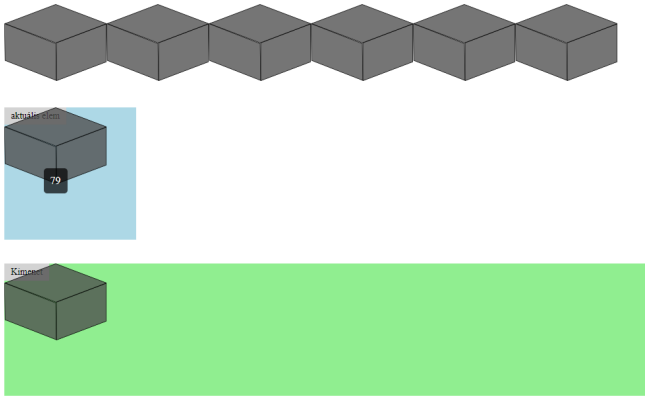
A megszámlálás során egy feltételnek eleget tevő elemeket számoljuk meg. Az összegzéstől annyiban tér el, hogy feltételhez kötjük az elemek vizsgálatát, valamint egy számlálót inkrementálunk, ha a feltétel igaz. Ismét nézzük a képeket először, hogyan is működik.
A feladatunk, hogy számoljuk meg a páros számokat a tömbünkben.



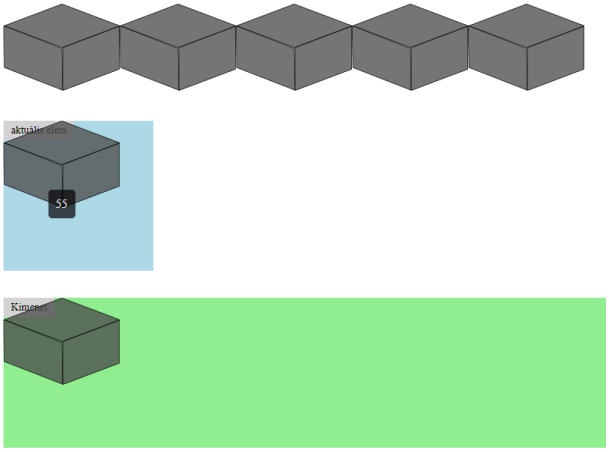
1. A már megszokott módon kezdünk a 8 dobozzal.



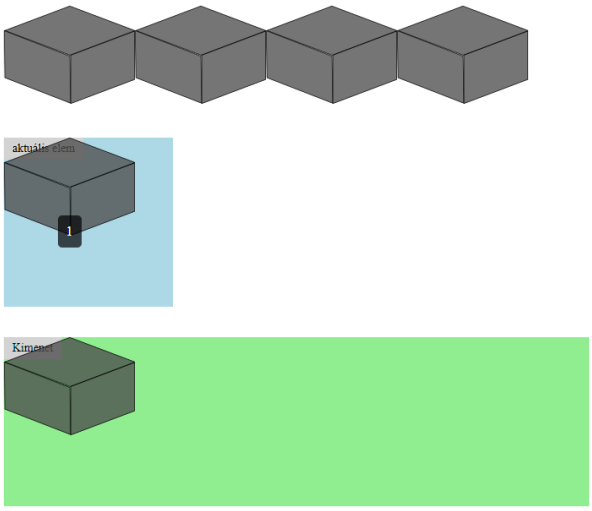
2. Levesszük az első dobozt, megnézzük mi van benne. Ez egy páros szám szerencsénkre ezért növeljük a számlálónkat, amiket most a dobozok száma fog reprezentálni



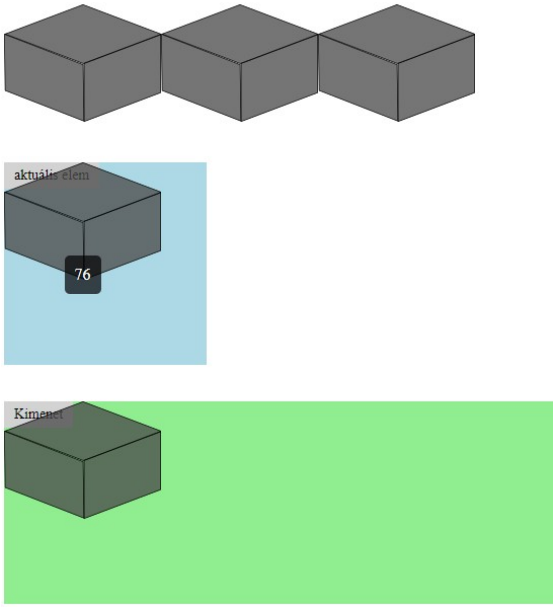
3. Levesszük a második dobozt, de ez most páratlan, ezért nem növeljük a számlálónkat.



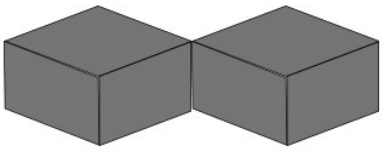
4. Ismételten páratlan számunk van, így nem növeljük a számlálónkat.

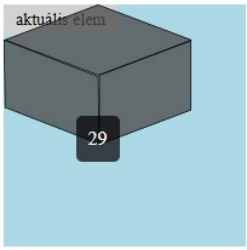


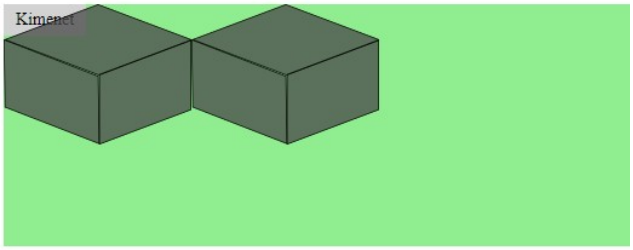
5. Páratlan szám jött elénk megint, ezért ismét nem növeljük a számlálónkat.



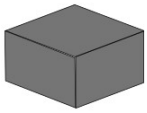
6. Végre van egy újabb páros számunk, ezért növeljük a számlálónkat.

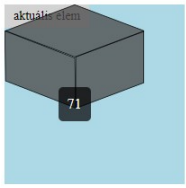


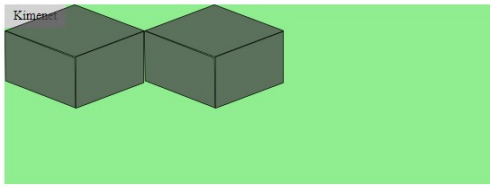




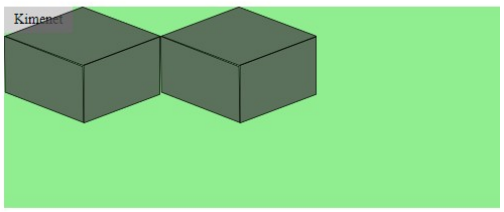
7. Páratlan számot vizsgálunk ismét, ez nincs hatással a kimenetünkre.







8. Páratlan számunk van, nem növelünk.



9. Az utolsó számunk is páratlan, szóval már tudjuk, hogy a kiindulási tömbünkben 2 páros szám szerepelt.

pszeudokód:
szamlalo = 0 // inicializáljuk a számláló változónkat, természetesen mivel nincs infónk, ez nulla lesz
ciklus i = 0 .. n – 1 // természetesen ez az algoritmus is addig ismétlődik, ahány elem van a tömbünkben
    ha t[i] % 2 == 0 akkor // a tömb aktuális eleme akkor páros, ha a 2-vel vett osztásának a maradéka 0
        szamlalo = szamlalo + 1 // ebben az esetben növeljük a számlálónkat
    ha vége
ciklus vége
ki szamlalo // természetesen az algoritmusunk végeztével visszatérünk a számlálóval

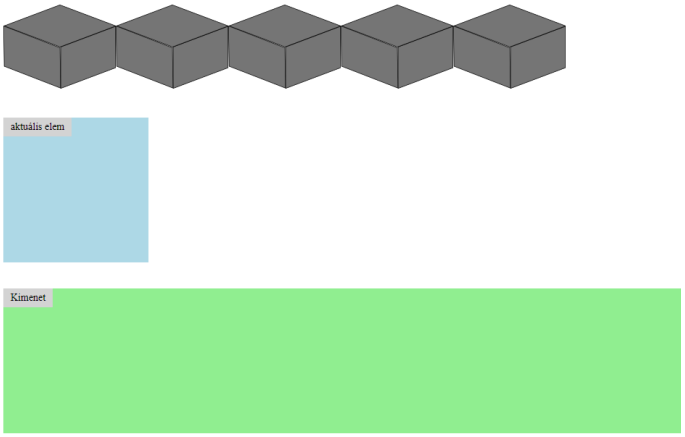
jskód:

```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
let count = 0;
for(const i in numbers){
    if(numbers[i] % 2 == 0){
        count ++;
    }

}

console.log(`A párps elemek száma: ${count}`)
```

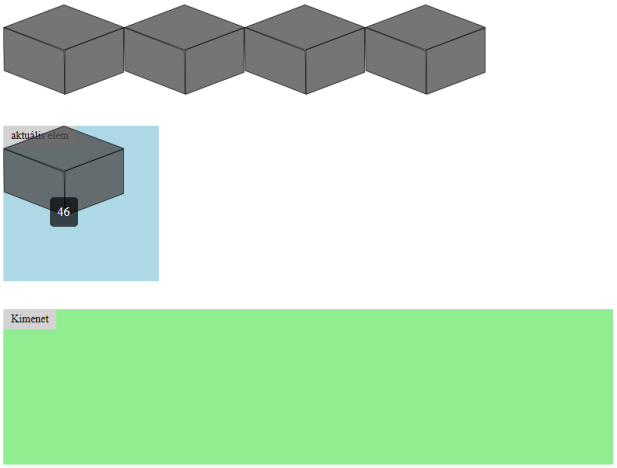
```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
let count = 0;
for(const number of numbers){
    if(number % 2 == 0){
        count ++;
    }

}

console.log(`A páros elemek száma: ${count}`)
```

```
const numbers = [86, 87, 75, 7, 3, 97, 25, 31]
let count = 0;
for(let i = 0; i< numbers.length; i++){
    if(numbers[i] % 2 == 0){
        count ++;
    }

}

console.log(`A páros elemek száma: ${count}`)
```

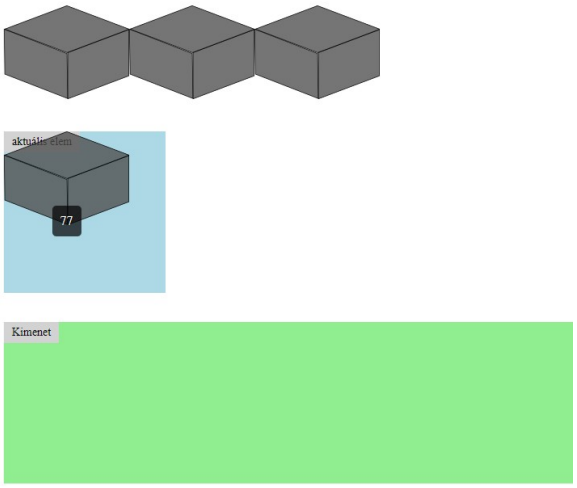
Kiválogatás
A kiválogatás a számlálástól annyiban különbözik, hogy nem egy számlálót növelünk, hanem egy tömbben gyűjtjük azokat az elemeket, amelyek megfelelnek a feltételünknek, amit megadunk. A szétválogatás és a kiválogatás megegyezik, a különbség csupán annyi, hogy a tömb minden elemét elhelyezzük egy tömbben.
A feladatunk, hogy kiválogassuk a páratlan számokat.



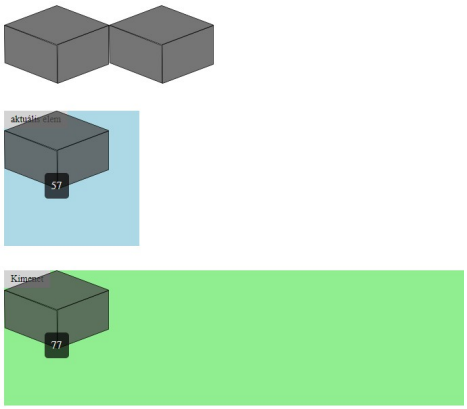
1. Megkapjuk a tömbünket. Ahogy eddig most is egyesével fogjuk vizsgálni mi van a dobozokban.



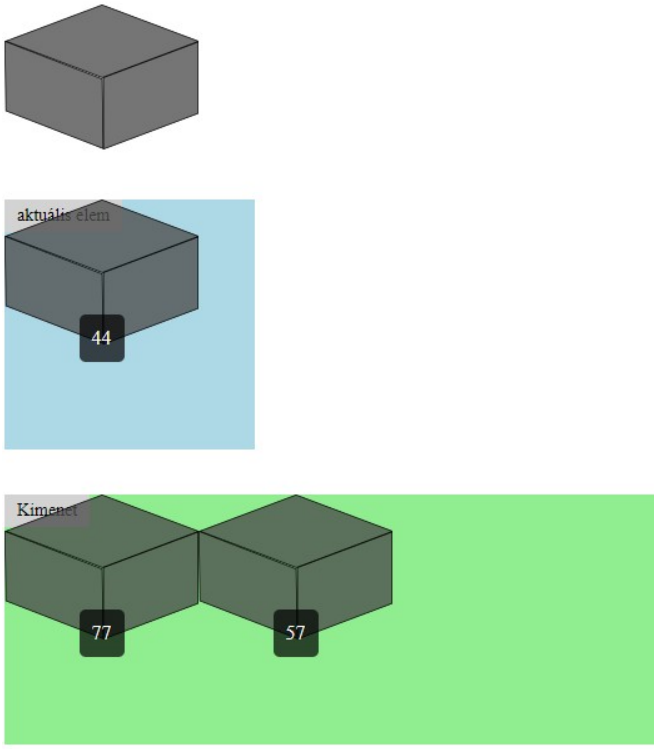
2. Az első elemünk egy páros szám így nem adjuk hozzá a kimeneti tömbünkhöz.



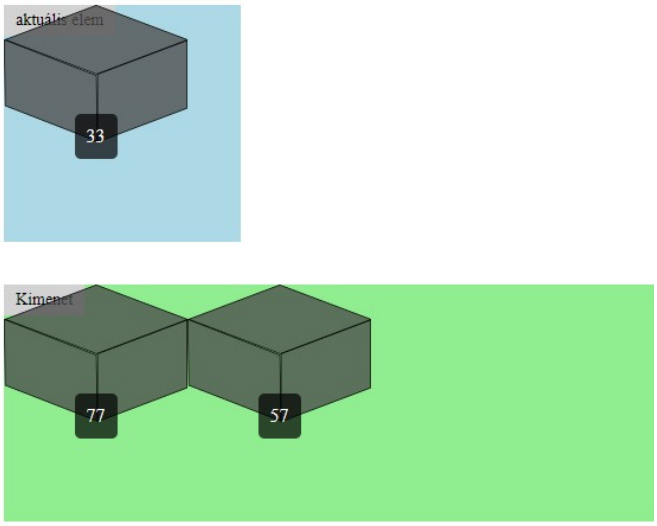
3. A második elemünk egy páratlan szám. Ezt hozzáadjuk a tömbünkhöz. Fontos, hogy nem egy számlálónk van, hanem egy eredménytömbünk, tehát egyszerűen a kimenethez rakjuk dobozzal és értékkel együtt.



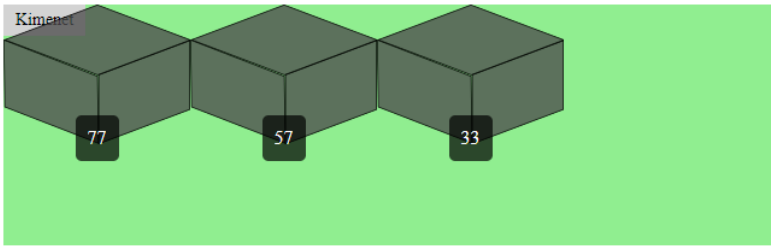
3. Ismét egy páratlan számmal találkoztunk, így ezt is hozzáadjuk a tömbünkhöz.



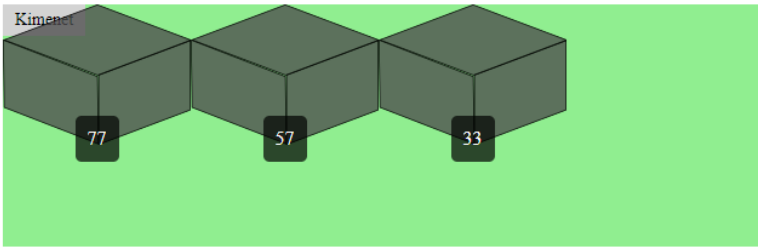
4. Sajnos megint egy páros számmal találkoztunk, így ezt a dobozt elengedjük.

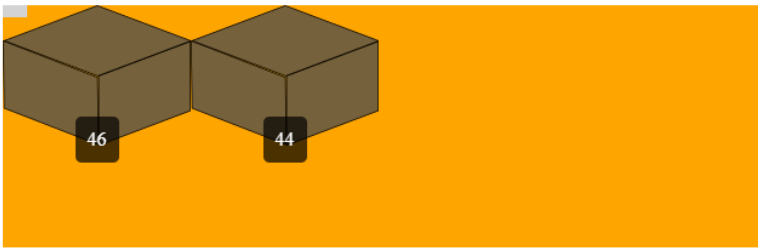


5. A bemeneti tömbünk utolsó eleme is páratlan, így azt is hozzáadjuk az eredményhez.



6. Ezen a képen látható az algoritmusunk kimenetele.
A kiválogatás egy formája a szétválogatás, ami lényegében ugyanezt teszi, csak két tömbbe válogatunk. A feladatunk megoldása így nézne ki, ha szétválogattuk a páros és páratlan számokat:





Hogy miért nem mindig válogatunk? Mert sokszor nem érdekel minket, hogy mi nem tartozik az eredményhalmazunkba, és így csak a memóriát tömjük tele.

pszeodokód:
j = 0 //az eredménytömbünk indexét inicializáljuk. Természetesen az eredménytömbünk indexelése is 0-tól kezdődik
ciklus i = 0 .. n – 1 // annyi iterációnk lesz, ahány elemű a bemeneti tömbünk
  ha a[i]%2 == 1 // ha a 2-vel vett osztásunk maradéka 1, akkor páratlan számmal van dolgunk
    b[j] = a[i] // mivel a bemeneti tömbünk aktuális eleme páratlan ezért belerakjuk az eredménytömbünkbe
    j = j + 1 // növeljük a az eredménytömbünk indexét értelemszerűen, hiszen nem szeretnénk felülírni az elemünket.
  ha vége
ciklus vége

js kód:

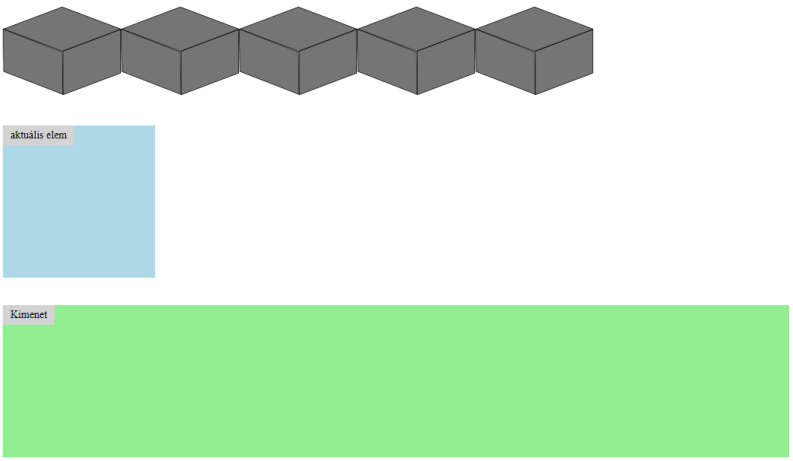
```
const numbers = [86, 87, 75, 7, 6, 92, 25, 31]
const result = [];
for(let i = 0; i< numbers.length; i++){
    if(numbers[i] % 2 == 0){
        result.push(numbers[i])
    }

}

console.log(`A páros elemek: ${result.join(', ')}`)
```

```
const numbers = [86, 87, 75, 7, 6, 92, 25, 31]
const result = [];
for(const number of numbers){
    if(numbers[i] % 2 == 0){
        result.push(number)
    }

}

console.log(`A páros elemek: ${result.join(', ')}`)
```

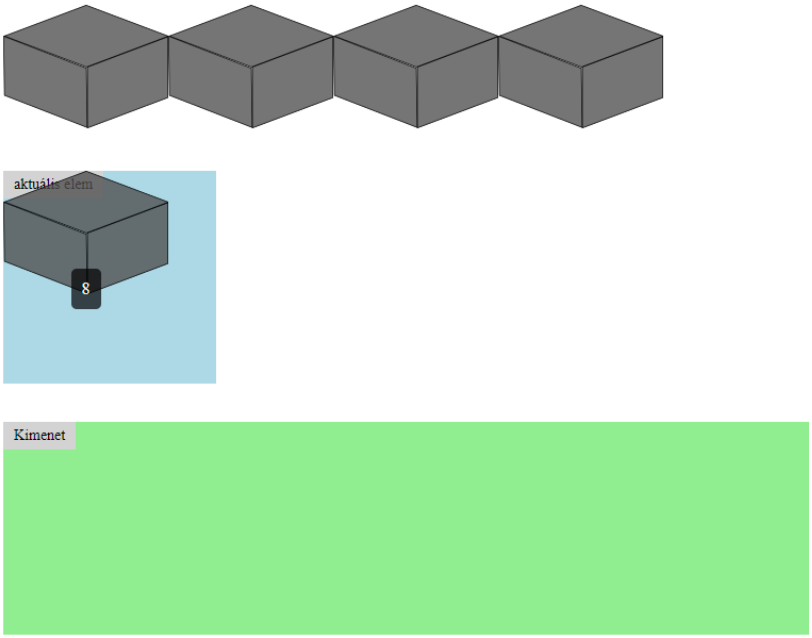
gyakorlás:
feladat1: Írjuk ki a bemeneti tömbünk páros elemeinek átlagát a számlálás, és összegzés algoritmus segítségével
feladat2: Válogassuk ki egy személy (név, életkor) tömbből azokat akiknek az életkora 2-vel osztható.

Rendezés

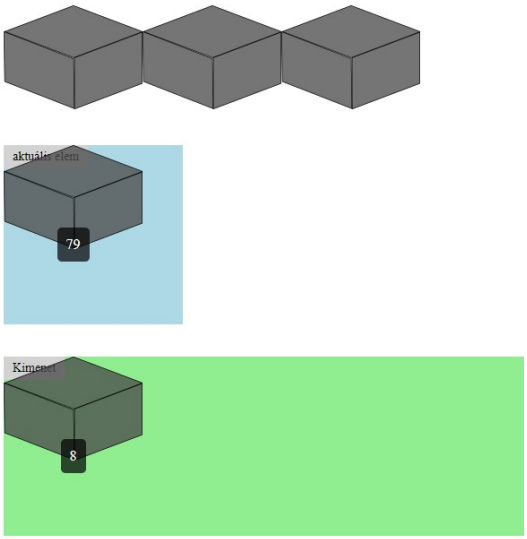
A rendezés az egyik legfontosabb algoritmusunk. A következő doksiban lesz meg erről szó, és további algoritmusokat fogunk megismerni. Jelenleg a legegyszerűbb algoritmussal fogunk foglalkozni, ami a rendezés cserével. Ahogyan azt látni is fogjuk, egy régi ismerősünket, a minimumkeresést fogjuk felhasználni a rendezéshez. A lényeg a következő: folyamatosan csökkentjük a bemeneti tömbünk méretét, és mindig megkeressük a minimum értéket. Kicsit furán hangzik. Feladatunk a bemeneti tömbünk csökkenő sorrendbe rendezése. Nézzük a képeket!



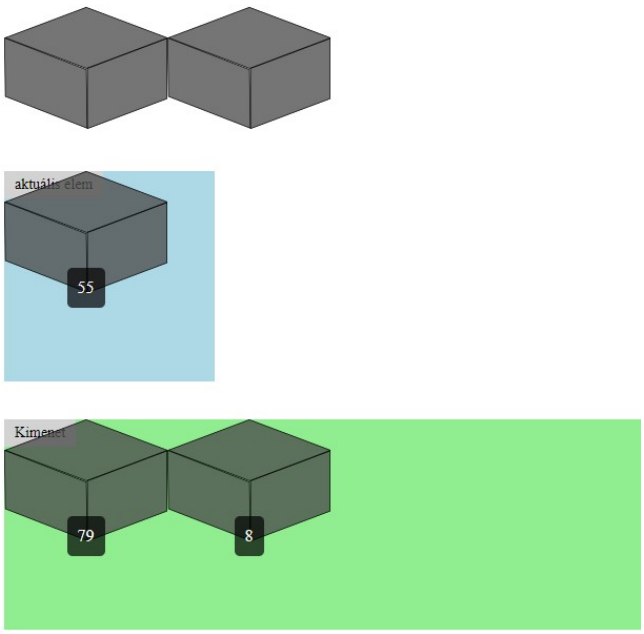
1. Felsorakozik 5 elemünk.



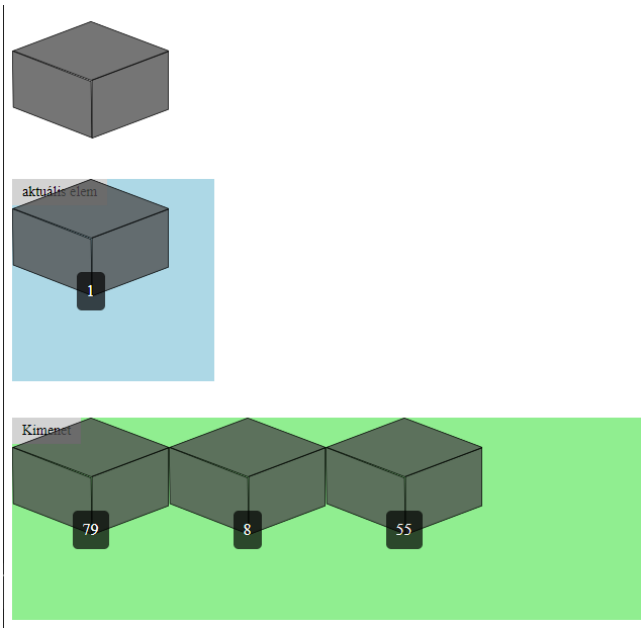
2. Azt mondjuk, hogy a 8-as elem a legnagyobb a tömbben. Hozzárakjuk a kimenethez.



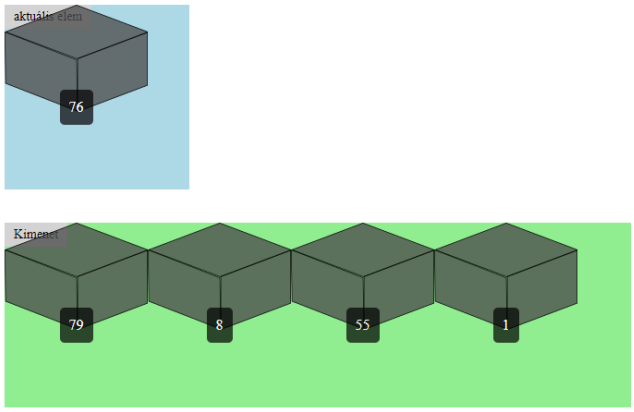
3. A 79 bizony nagyobb mint a 8. így a kettő helyet cserél.



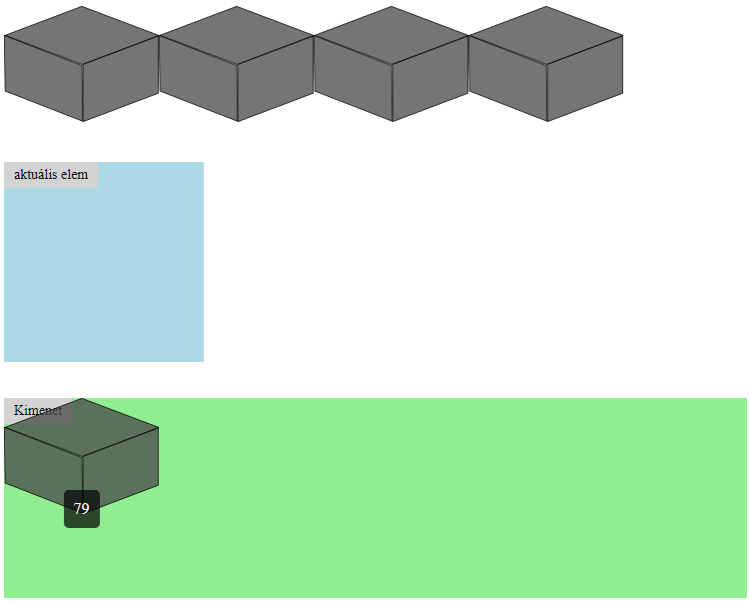
4. Miután a 79 lett az első, ahhoz hasonlítjuk az új elemünket az 55-t. Mivel nem nagyobb, marad a helyén.



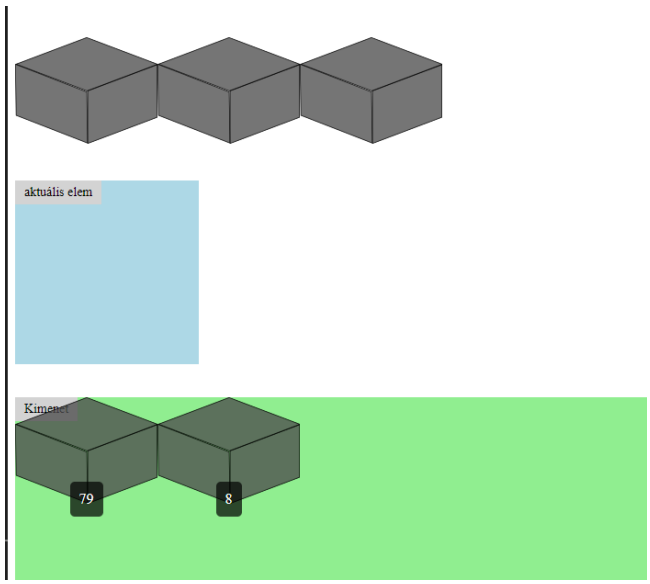
5. Az 1 sem nagyobb 79-nél, az is marad a helyén.



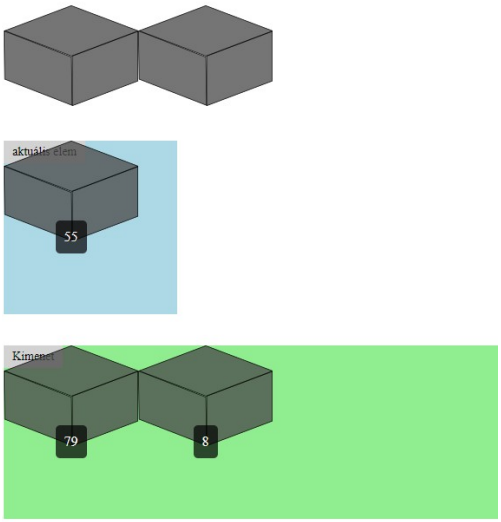
6. A 76 sem nagyobb a 79-nél, így az is marad a helyén. Végigértünk a tömbön, megvan az első maximumunk. Az algoritmusunk viszont még nem ért véget.



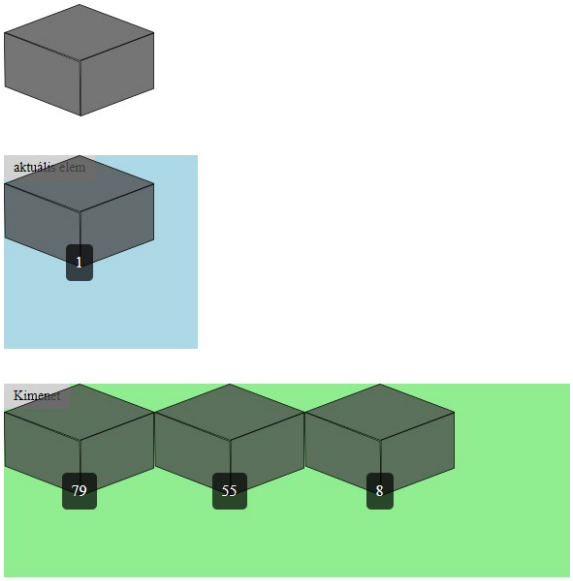
7. A megmaradt 4 elemünkön fogunk most végigmenni. Mintha az a 79 nem is létezne, és a 4 elemünk maximumát keressük.



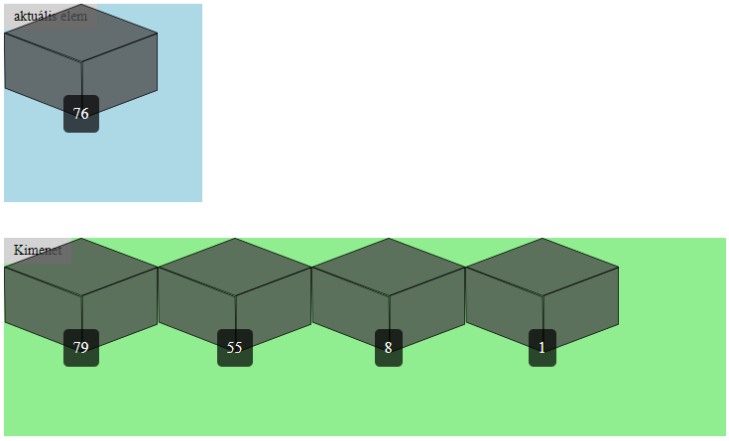
8. Az első elemet egyből ki is jelöljük maximumnak. Emlékeztetőül, a 79-et és a 8at kicseréltük korábban.



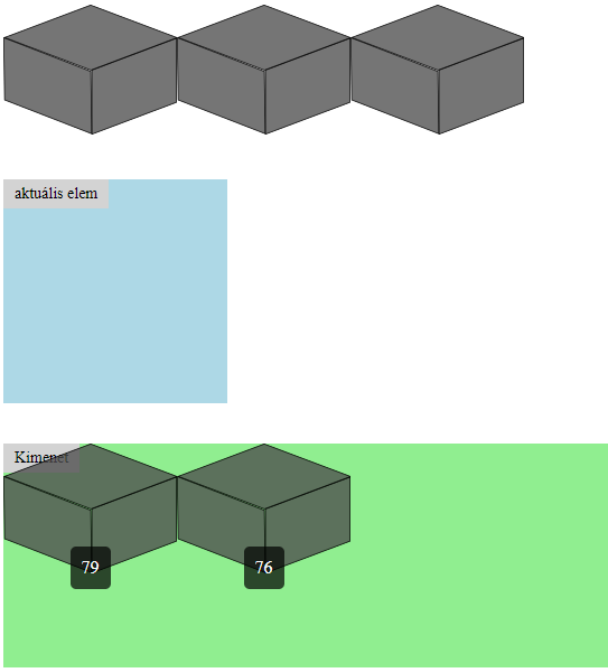
9. Az 55-t most a második elemhez hasonlítjuk, természetesen nagyobb, így cseréljük.



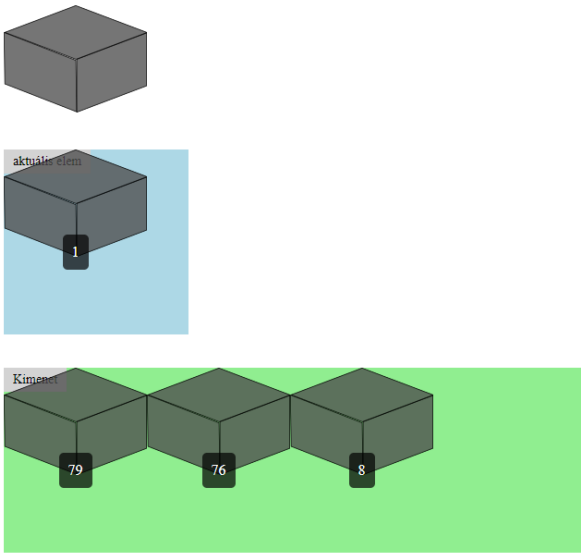
10. Most az 1-et az 55-höz hasonlítjuk, nyilván nem nagyobb, így marad a helyén.



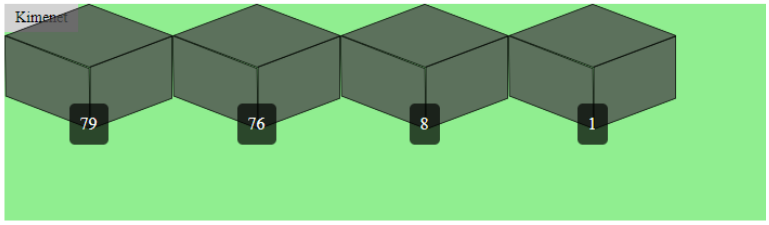
11. A 76 nagyobb mint az 55, így azokat cseréljük.



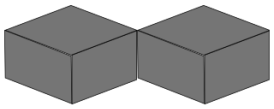
12. Végigértünk a tömbön, már csak 3 elemünk maradt, értelemszerűen az elsőt rögtön le is tehetjük, és most már a 3. elemhez fogunk hasonlítgatni.

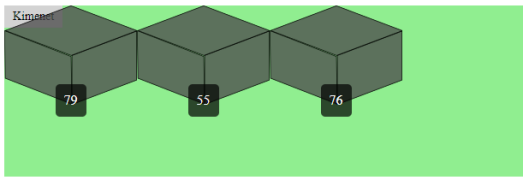


13. Az 1 természetesen nem nagyobb a 8. Így az 1 marad a helyén.

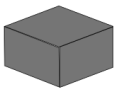


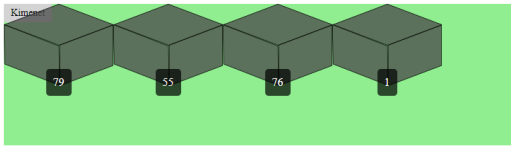
14. Az 55 nagyobb a 8-nál, így a kettőt kicseréljük.



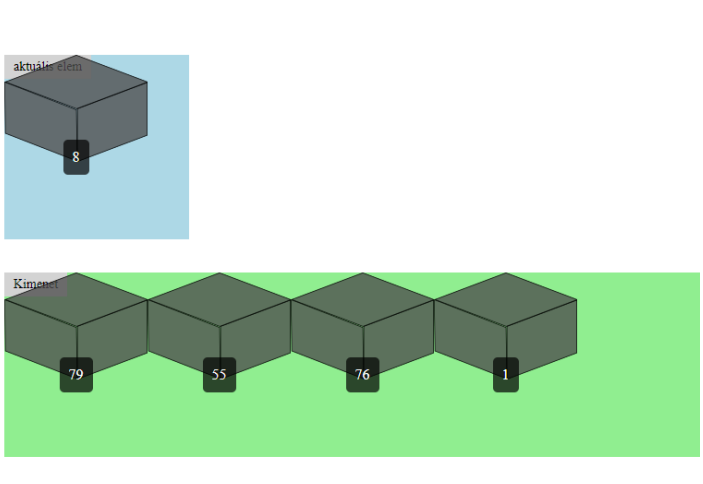


15. A megmaradt két elem között kell maximumot keresnünk. Az elsőt természetesen egyből le is tudjuk rakni.

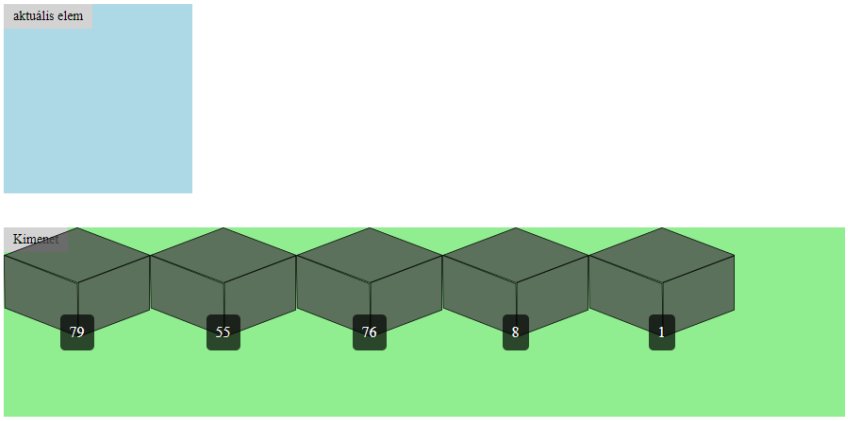




16. Pechünkre ugyan az 1-et raktuk le, de már rövidebb szerencsére a maximum, hiszen a másikban tudjuk mi is lehet.



17. A 8 nagyobb mint az 1, így csere lesz.



18. Az utolsó két elem cseréjével véget is ért az algoritmusunk, a tömbünk rendezetté vált.

A lépések számából is talán látszik, hogy ez nem egy túl optimális eljárás.

pszeudokód:
Ciklus i := 0 .. n-2 // az utolsó előtti elemik megyünk a tömbünkön
  Ciklus j := i+1 .. n-1 //Minden egyes lépésnél elkezdjük összehasonlítani a tömb megmaradt elemeivel. Ezzel indokolható az is, hogy miért az utolsó előtti elemig megy a külső ciklus
    Ha t[i] < t[j] akkor // Ha a jelenleg hasonlítgatott elemünk kisebb mint az amivel hasonlítjuk
      tmp = t[i] // kimentjük az eredeti elemet (amit hasonlítunk) egy ideiglenes változóba
      t[i] = t[j] // kicseréljük azzal amivel hasonlítjuk
      t[j] = tmp // az ideiglenes változóból kivesszük az eddig legnagyobbnak vélt elemet
    Elágazás vége
  Ciklus vége
Ciklus vége

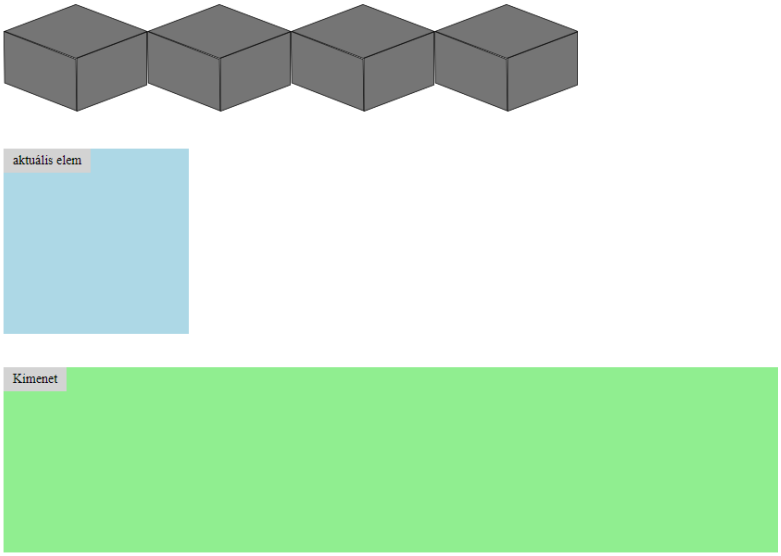
js kód:

```
const numbers = [86, 87, 75, 7, 6, 92, 25, 31]

for(let i = 0; i< numbers.length-1; i++){
    for(let j = i+1; j < numbers.length; j++){
        if(numbers[i] < numbers[j]){
            const tmp = numbers[i]
            numbers[i] = numbers[j]
            numbers[j] = tmp
        }

    }
}

console.log(`A rendezett elemek: ${numbers.join(', ')}`)
```

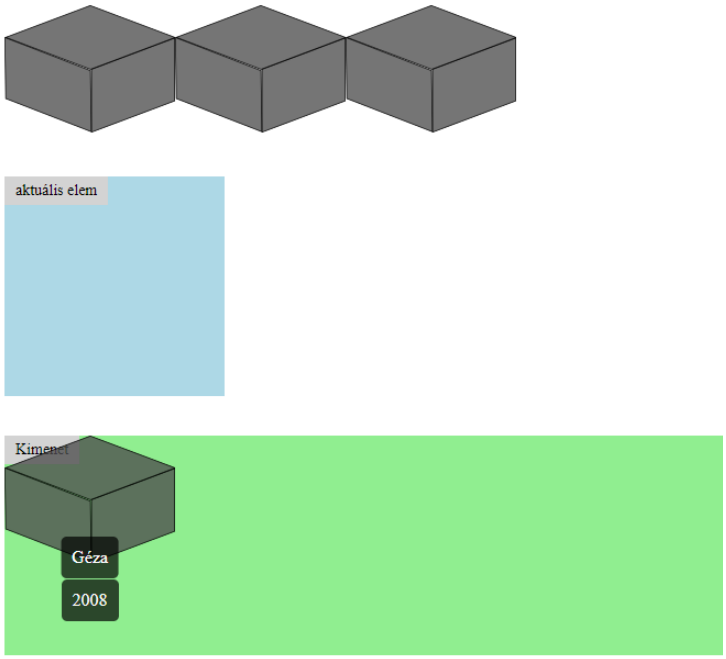
Nem csak számok

A számtömbös példák után nézzünk meg egy olyan példát is, ahol a dobozokban személyek szerepelnek. Láthatjuk, hogy különösebb különbség nincs az algoritmusban, csak a bemeneti tömb elemei változtak meg.

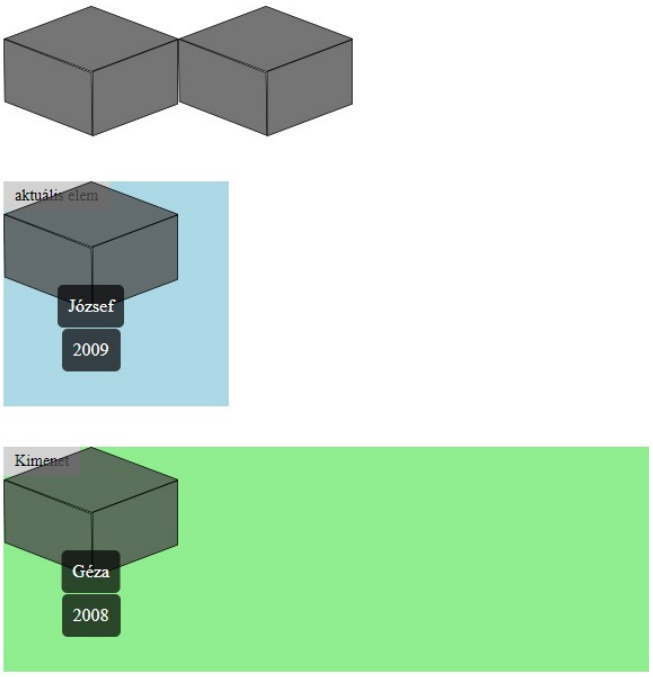
Feladatunk, hogy egy személyeket (név, születési év) tartalmazó tömbben keressük meg a legidősebb embert.



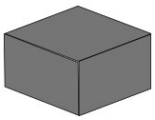
1. A bemeneti tömbünk az algoritmus szempontjából változatlan. Mint azt tudjuk, a legidősebb embert úgy fogjuk megtalálni, hogy a legkisebb születési évvel rendelkező embert keressük. Klasszikus minimumkeresés feladat, első lépésben a tömbünk első elemét vesszük a minimumnak.



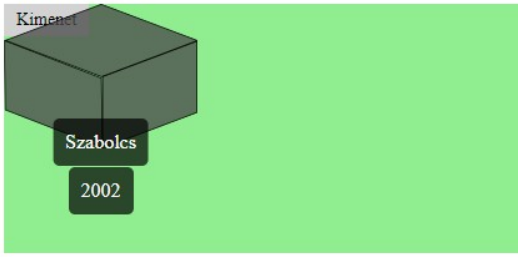
2. Most, hogy megadtuk az első elem értékét a kimeneti változónknak, neki is állhatunk a vizsgálatnak.



3. József 2009-ben született, a 2009 nagyobb 2008-nál, tehát a kimenetünk nem változik.



4. Szabolcs 2002-ben született, ez egy kisebb szám 2008-nál, ezért cseréljük.



5. Balázs később született Szabolcsnál, ezért a kimeneti objektumunk nem változik. A tömbünkben Szabolcs volt a legidősebb doboz. Láthatjuk, hogy az algoritmusban nincs különösebb változás, csupán a dobozoknál megjelent egy újabb címke, és tulajdonságokat hasonlítunk egymáshoz.

jskód:

```
const personList = [
    { name: 'Géza', birthYear: 2008 },
    { name: 'József', birthYear: 2009 },
    { name: 'Szabolcs', birthYear: 2002 },
    { name: 'Balázs', birthYear: 2004 }];


let old = personList[0];
for (let i = 0; i < personList.length - 1; i++) {
    if(personList[i].birthYear < old.birthYear){
        old = personList[i];
    }
}

console.log(`A legidősebb személy: ${old.name}`)
```

gyakorlás
feladat1: Rendezzünk ár szerint csökkenő sorrendbe egy árukat (megnevezés, típus, ár) tartalmazó tömböt.
feladat2: Összegezzük a kiadásokat (név, hónap, összeg) hónapok alapján.